

LIBRARY MANAGEMENT SYSTEM

NAME=VANSH KAPOOR

REGISTRATION NUMBER=26BCE11345

COURSE=INTRODUCTION TO PROBLEM SOLVING

COURSE CODE=CSE1021

SUBMITTED TO=E NIRMALA

This technical report details the software engineering lifecycle, architectural patterns, and testing suite for a Python-based Library Management System. Featuring full decoupled persistence, object-oriented domain modeling, and robust CLI interactions.

PROJECT TITLE

Library Management CLI

LANGUAGE / ENGINE

Python 3.10+ Standard Library

ARCHITECTURE

3-Tier Service/Storage Pattern

PERSISTENCE LAYER

JSON Document File System

AUTHOR / LEAD

Software Engineering Team

DOCUMENT DATE

October 2026

1. INTRODUCTION & OVERVIEW

System Context

Information management systems form the operational backbone of modern academic and public libraries. The **Library Management System (LMS)** developed in this project is a lightweight, terminal-based software solution engineered to digitize book cataloging, inventory tracking, and lending operations.

While enterprise resource planning (ERP) platforms cater to large institutional libraries, small community collections, school media centers, and departmental libraries require zero-configuration, dependency-free software that functions efficiently without heavy infrastructure dependencies.

2. PROBLEM STATEMENT

Engineering Need

Manual cataloging using physical registers or unvalidated spreadsheets introduces significant operational bottlenecks:

Key Operational Inefficiencies

- **State Inconsistency:** High risk of double-issuing titles due to real-time status tracking failures.
- **Lookup Latency:** Time-consuming manual search processes to verify book availability.
- **Data Loss & Corruption:** Susceptibility to register damage, loss, or human entry errors.
- **Software Over-engineering:** Enterprise LMS platforms demand complex database installations (SQL/NoSQL) and network configuration for simple offline workflows.

3. SYSTEM SCOPE & BOUNDARIES

Project Boundaries

✓ In-Scope Capabilities

- Modular 3-tier Python software architecture.
- Persistent state sync via formatted JSON disk storage.
- CRUD inventory tracking (Add, View, Lookup).
- Circulation logic (Issue and Return workflows).
- Input validation and terminal exception catching.

✗ Out-of-Scope Elements

- Multi-user concurrent network server.
- Relational SQL database engine integration.
- Graphical User Interface (GUI) or Web client.
- Fine accumulation or overdue penalty calculation.

4. FUNCTIONAL REQUIREMENTS

Specification

The system behavior is governed by six core functional requirements mapped directly to user workflows:

Req ID	Module	Requirement Description
FR-01	Book Registration	The system shall allow users to register a new book with a unique alphanumeric ID, Title, and Author. Default availability must be set to `True`.
FR-02	Catalog Display	The system shall render a complete list of all registered books displaying ID, Title, Author, and active status (`Available` / `Issued`).
FR-03	Target Lookup	The system shall execute a linear search by Book ID and output exact book metadata or an error message if missing.
FR-04	Circulation: Issue	The system shall mark an available book as `Issued` (`available = False`). Duplicate check-outs must be rejected with an alert.
FR-05	Circulation: Return	The system shall mark an issued book as `Available` (`available = True`). Returning unissued books must be rejected.
FR-06	Data Persistence	The system shall automatically load records from disk upon startup and flush changes to `data/books.json` upon state mutation.

5. NON-FUNCTIONAL REQUIREMENTS

Quality Attributes

NFR-01: Performance & Response

All CLI operations (search, catalog view, checkout) must execute within less than 50 milliseconds under a 1,000-book record load.

NFR-02: Maintainability & Quality

Code must strictly adhere to PEP 8 style standards, utilize explicit type hints, and feature modular class separation.

NFR-03: Data Reliability

JSON persistence operations must implement auto-creation of missing data directories and handle corrupted files gracefully.

NFR-04: Portability & Dependencies

The application must run natively across Windows, macOS, and Linux using only the standard Python 3 runtime library.

6. SYSTEM ARCHITECTURE

The software utilizes a clean **3-Tier Layered Architecture** separating Presentation, Business Logic, and Persistence layers. This separation ensures loose coupling, simplified unit testing, and isolated data management.

7.1 USE CASE DIAGRAM

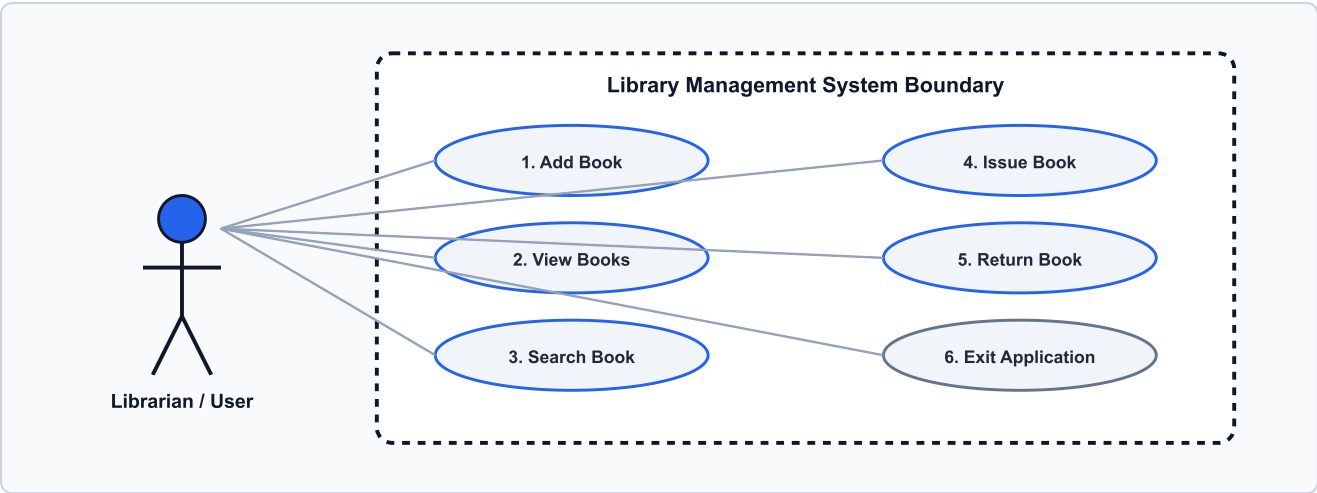


Figure 7.1: System Use Case Diagram detailing user interactions with application boundary options.

7.2 WORKFLOW / ACTIVITY FLOW DIAGRAM

The operational flowchart below models the execution control flow of the CLI application, from initial menu presentation and input validation to service execution and persistence sync.

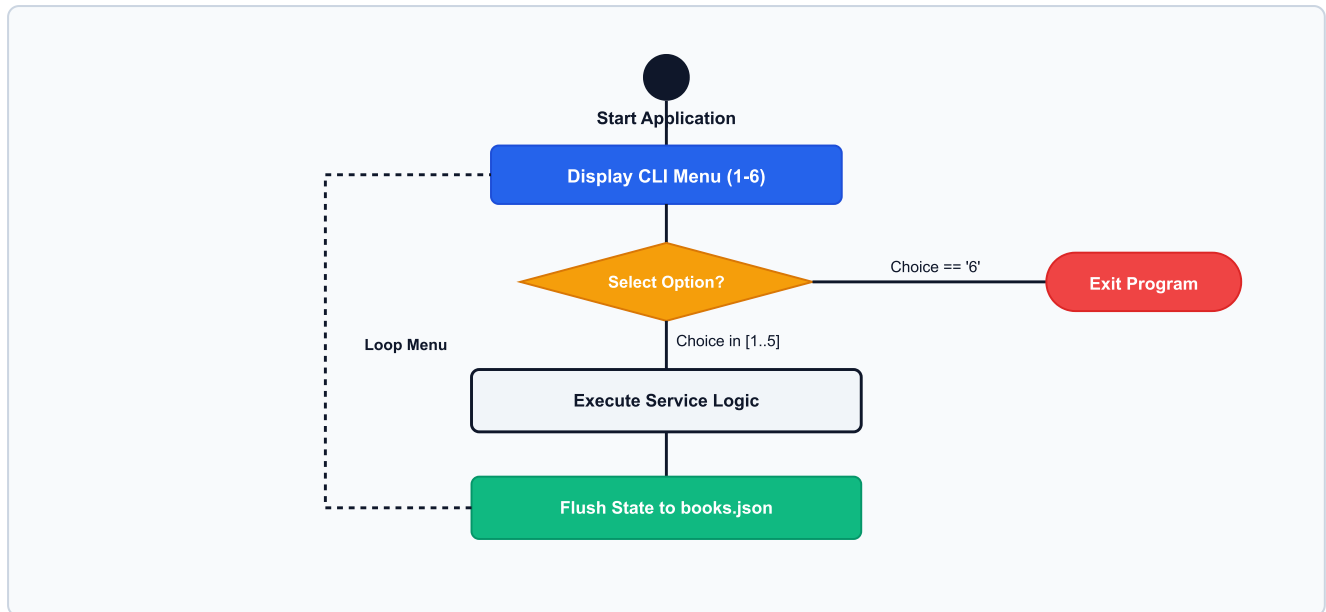


Figure 7.2: Application operational workflow showing interactive menu control loop.

7.3 SEQUENCE DIAGRAM

The UML sequence diagram below traces object message exchanges across architectural layers during an **Issue Book** (`issue_book`) invocation.

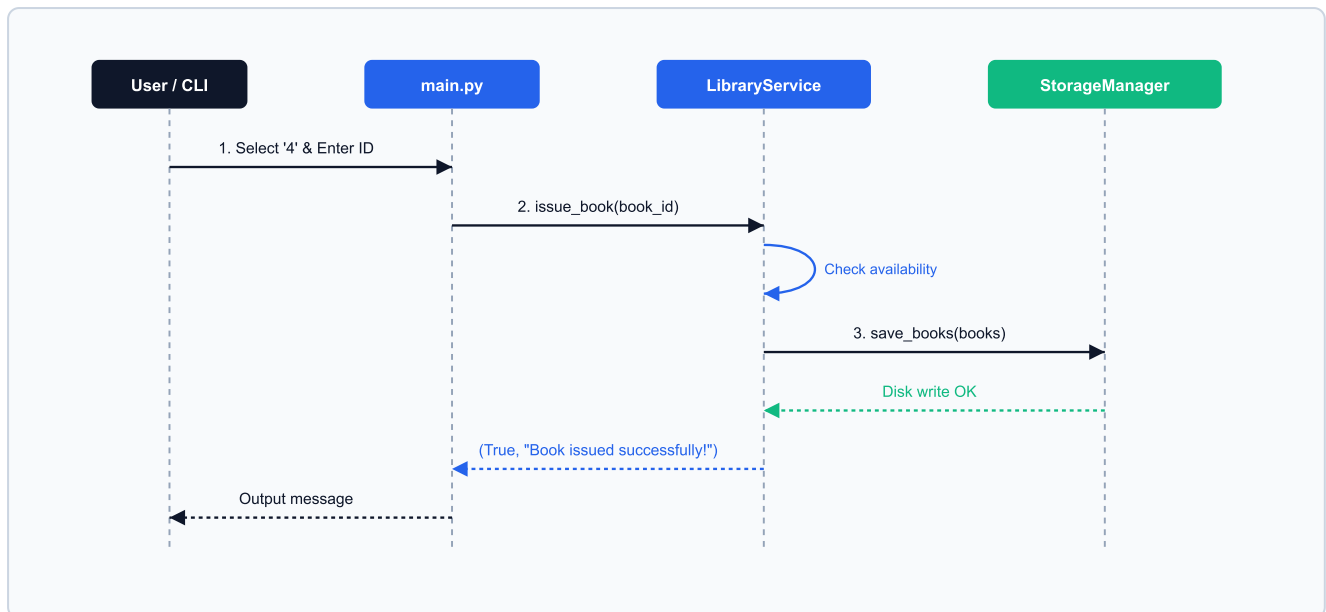


Figure 7.3: Sequence diagram detailing the book issue lifecycle across system layers.

7.4 CLASS / COMPONENT DIAGRAM

The UML class diagram specifies core classes (`Book`, `StorageManager`, `LibraryService`), their attributes, visibility accessors, and dependency injections.

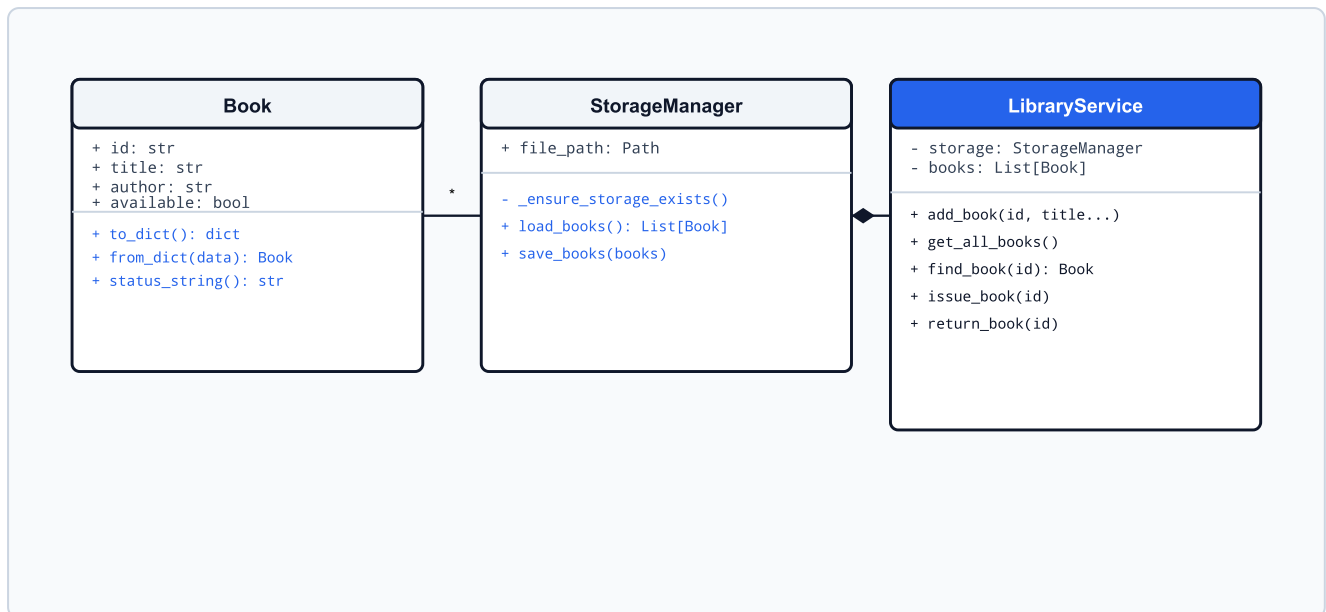
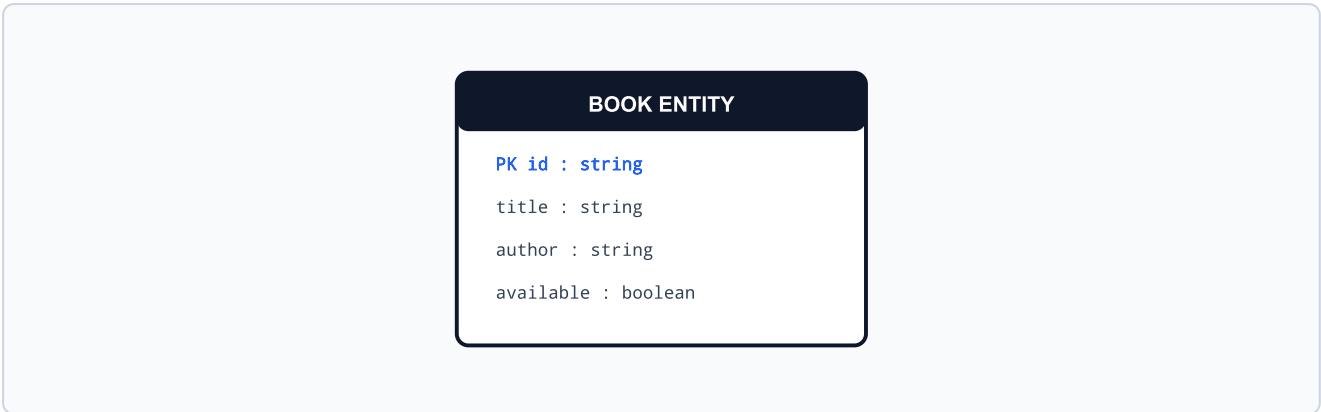


Figure 7.4: Object-oriented class architecture showing data models and service associations.

7.5 ENTITY-RELATIONSHIP (ER) & PERSISTENCE MODEL

Data Model

Although utilizing flat JSON file persistence, the data model strictly mirrors a relational entity schema. The diagram below illustrates the entity structure alongside its JSON storage representation.



DATA DICTIONARY TABLE

Data Schema

Field Name	Data Type	Constraint	Description
`id`	String	Primary Key / Unique	Alphanumeric unique identifier for each book record.
`title`	String	Non-Null	Full title designation of the registered book.
`author`	String	Non-Null	Name of the author or writing contributor.
`available`	Boolean	Default: True	State flag indicating availability (`True` = Available, `False` = Issued).

8. DESIGN DECISIONS & TRADE-OFF RATIONALE

Architecture Rationale

Key engineering decisions were made during project development to balance complexity, portability, and performance:

Design Decision	Selected Approach	Architectural Rationale
Data Storage Engine	JSON File Persistence (`data/books.json`)	Eliminates external database dependencies (e.g. SQLite/MySQL), ensuring 100% zero-configuration execution across any system with Python installed.
Architecture Pattern	3-Tier Decoupled Layering (`models`, `storage`, `library`, `main`)	Replaces monolithic procedural scripts with distinct class responsibilities, facilitating isolated unit testing and clean maintenance.
Dependency Strategy	Standard Library Only (`json`, `pathlib`, `typing`)	Guarantees instant setup without needing `pip install` or virtual environment dependency overhead for evaluators.
Lookup Algorithm	Linear Scan ($O(N)$ search over list)	Provides optimal low-overhead performance for small-to-medium library catalogs (< 10,000 items) with minimal memory overhead.

9. IMPLEMENTATION DETAILS

Core Code Architecture

Below is an excerpt of the core status validation logic in `LibraryService` handling circulation control:

```
def issue_book(self, book_id: str) -> Tuple[bool, str]: """Changes a book's availability status to Issued after state validation.""" book = self.find_book(book_id) if not book: return False, "Error: Target Book ID not found in catalog." if not book.available: return False, "Conflict: Book is already issued to another patron." book.available = False self.storage.save_books(self.books) # Atomic persistence update return True, "Success: Book issued successfully!"
```

10. EXECUTION RESULTS / CLI SCREENSHOTS

Verification

The following simulated terminal session illustrates typical execution workflows and interactive menu outputs:

```
bash — python main.py

===== LIBRARY MANAGEMENT SYSTEM ===== 1. Add Book 2. View Books 3. Search Book 4. Issue
```

11. TESTING APPROACH & VALIDATION SUITE

Quality Assurance

Test Case ID	Target Module	Input Test Condition	Expected Result	Status
TC-01	`LibraryService.add_book`	Duplicate Book ID ("101")	Rejection message: "Book ID already exists"	PASS
TC-02	`LibraryService.issue_book`	Issue already issued book	Rejection message: "Book is already issued"	PASS
TC-03	`StorageManager.load`	Missing `data/books.json` file	Auto-creation of directory and default empty list	PASS
TC-04	`main.py` CLI Input	Invalid menu selection ("99")	Warning message & menu prompt loop refresh	PASS

12. CHALLENGES FACED

Engineering Hurdles

During development, handling unexpected program terminations without corrupting the persistent JSON storage required implementing safe file writing patterns (`open(..., 'w')` flushing). Additionally, handling missing target directories across cross-platform OS environments was solved using Python's modern `pathlib.Path` standard module.

13. KEY LEARNINGS & TAKEAWAYS

Reflections

- **Decoupling Architecture:** Separating storage logic from CLI user code significantly improves testability and readability.
- **Type Safety in Python:** Explicit type annotations (`Tuple[bool, str]`, `Optional[Book]`) prevent runtime type mismatch bugs.
- **Defensive File I/O:** Always anticipating missing files or malformed JSON payloads ensures continuous application stability.

14. FUTURE ENHANCEMENTS ROADMAP

Expansion Plan

Phase 1: SQLite & Search

Migrate from JSON storage to SQLite for relational indexing and implement fuzzy title/author search capabilities.

Phase 2: Web API & REST

Expose core library operations via FastAPI endpoints and integrate web-based UI dashboards using HTML5/React.

15. REFERENCES

Citations

1. Python Software Foundation. (2026). *Python 3 Standard Library Documentation: 'json' & 'pathlib' modules*. <https://docs.python.org/3/>
2. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
3. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.